MCMP117 · WEEK 4

Fundamentals of Operating Systems

Week 4 · Threads & CPU Scheduling Basics

How can one program do several things at once — and who decides what runs next?

Electronical Communication Technology Programme · Vocational School · Istanbul Okan University

LAST WEEK

Week 3 in Four Ideas

The process

Its memory: text (code), data, bss, heap and stack — plus CPU state and open files.

Five states

new → ready → running → waiting → terminated. Only one process per core is running at a time.

The PCB

The kernel's record of a process: state, PC, registers, memory info, open files, scheduling info.

Context switch

Save the PCB of one process, load another's. fork() creates, exec() replaces, wait() collects.

Today: one process may contain several threads of execution — and the OS needs a rule for which one runs next.

Today's Agenda

1. **Threads** Why threads? What threads share and what they keep private; benefits.
2. **Multicore** Concurrency vs. parallelism; Amdahl's law with a worked example.
3. **Thread models** User vs. kernel threads; many-to-one, one-to-one, many-to-many; Pthreads.
4. **Scheduling basics** CPU–I/O bursts, ready queue, preemption, the dispatcher.
5. **Criteria** Utilisation, throughput, turnaround, waiting and response time — computed.

PART 1

Threads

One process, several threads of execution.

Why Threads? A Web Server Example

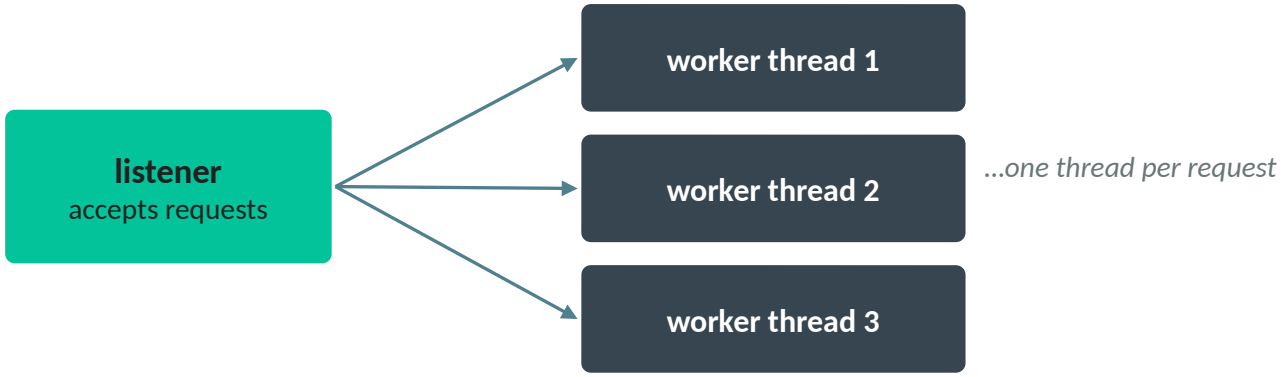
A web server receives thousands of requests per second. Each request needs a little CPU and a lot of waiting (disk, network).

Single-threaded server



While request 1 waits for the disk, requests 2–4 just wait in line.

Multithreaded server



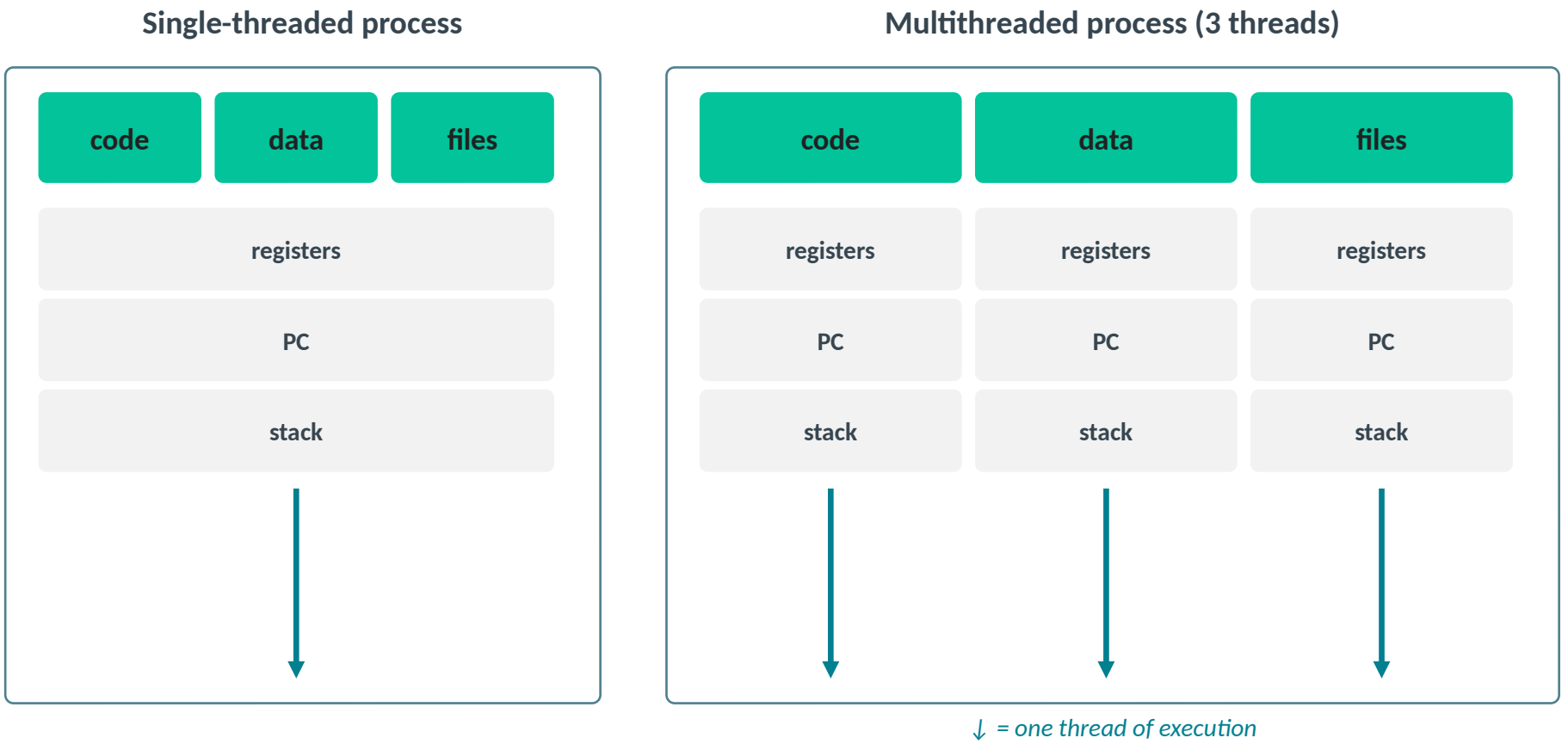
Same idea, everywhere

- **Word processor:** one thread reads keystrokes, one redraws the screen, one checks spelling, one autosaves.
- **Browser:** each tab loads, decodes images and runs scripts at the same time.
- **Game:** rendering, physics, sound and network in separate threads.
- All threads work on the same document / page / world — so they must share memory.

DEFINITION

A Thread Is a Unit of CPU Execution

A thread has its own thread ID, program counter, register set and stack. It shares everything else with the other threads of its process.



Note

A traditional ("heavyweight") process is simply a process with exactly one thread.

The PCB of Week 3 now splits: per-process part + one small block per thread.

What Do Threads Share? What Is Private?

Item	Threads of one process…	Why
Code (text section)	share	All threads run functions of the same program.
Global data & heap	share	A global variable or a malloc'ed array is visible to every thread.
Open files, PID, user ID	share	Resources belong to the process, not to a thread.
Program counter & registers	private	Each thread is at its own place in the code.
Stack (local variables)	private	Each thread has its own chain of function calls.
Thread ID, scheduling state	private	The OS schedules each thread separately.

Sharing is the whole point — and the danger: two threads changing the same variable at the same time give wrong results (race condition, Week 5).

Four Benefits of Multithreading

1. Responsiveness

If one thread blocks or does a long computation, the others keep going — the user interface does not freeze.

2. Resource sharing

Threads share memory and files automatically. Processes need extra mechanisms (shared memory, pipes) to cooperate.

3. Economy

Creating a thread and switching between threads are much cheaper than for processes: no new address space is needed.

4. Scalability

On a multicore CPU, threads of one process can run in parallel on different cores. A single-threaded process uses one core only.

Next slide: we measure benefit 3 — how much cheaper is a thread than a process?

HANDS-ON

Example 1: Is a Thread Really Cheaper?

```c
#define N 5000
void *nothing(void *arg) { return arg; }

/* 1) N processes */
for (int i = 0; i < N; i++) {
    pid_t pid = fork();            /* copy the process */
    if (pid == 0) _exit(0);        /* child exits      */
    waitpid(pid, NULL, 0);
}
/* 2) N threads */
for (int i = 0; i < N; i++) {
    pthread_t t;
    pthread_create(&t, NULL, nothing, NULL);
    pthread_join(t, NULL);
}
/* full program: code/create_cost.c */
```

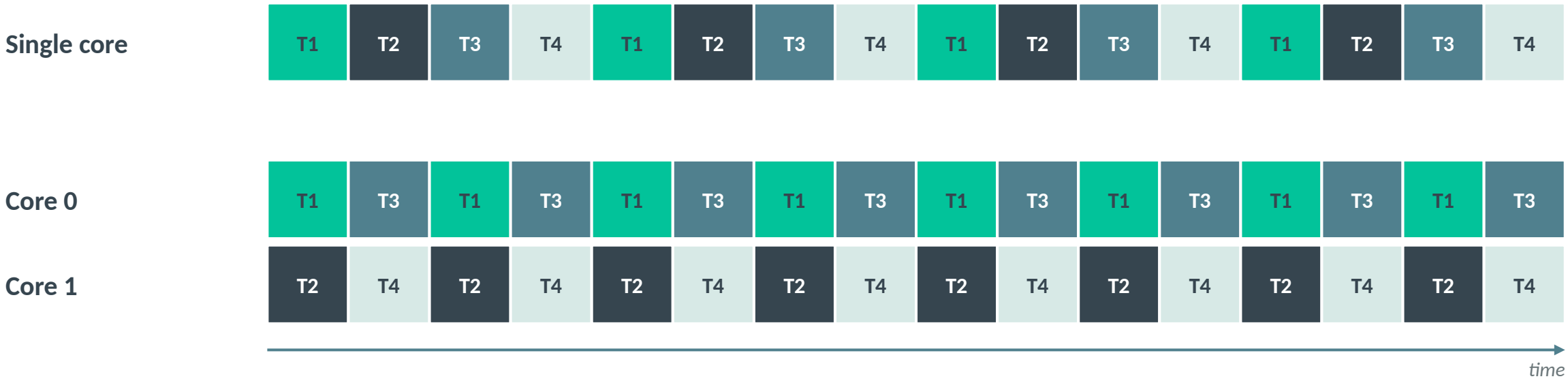
The experiment

- Create 5000 processes (fork + wait), each exits immediately.
- Then create 5000 threads (pthread_create + join) that do nothing.
- Measured on a 2-core Linux virtual machine:

```
$ ./create_cost
fork + wait          :  205.5 us per process
pthread_create + join:   51.9 us per thread
a thread is about 4 times cheaper to create
```

Concurrency vs. Parallelism

Four threads T1–T4 over time. Same program, two different machines.



Concurrency

Tasks make progress by taking turns — even on ONE core.

Parallelism

Tasks execute at the SAME instant — needs ≥ 2 cores.

Programming for Multicore: Five Challenges

Identify tasks

Find parts of the work that can run independently.

Balance

Give each core a similar amount of work, or some cores sit idle.

Data splitting

Divide the data so each thread works on its own part.

Data dependency

If task B needs A's result, B must wait: synchronisation.

Testing

Many possible interleavings: bugs appear only sometimes.

Data parallelism: same operation on different parts of the data (our array sum). Task parallelism: different operations at the same time (word-processor threads).

Amdahl's Law: The Limit of Adding Cores

Part of every program is serial (cannot be split). That part does not get faster with more cores.

$$\text{speedup} \le \frac{1}{S + (1 - S) / N}$$

S = serial fraction of the work, N = number of cores



Reading the formula

- With 1 core the program takes time S + (1 − S) = 1.
- With N cores only the parallel part shrinks, to (1 − S)/N.
- **Limit:** even with infinitely many cores, speedup ≤ 1/S.
- 25 % serial ⇒ never more than 4 × faster, no matter how many cores you buy.

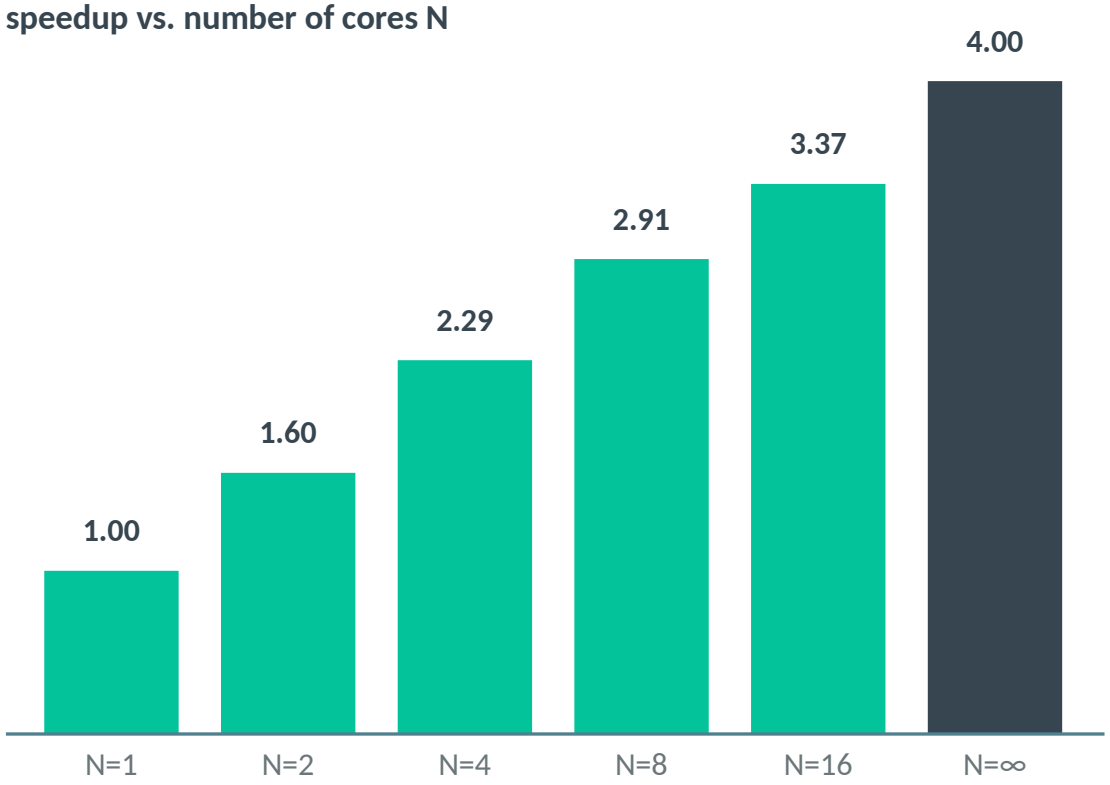
Worked Example: 75 % Parallel, 25 % Serial

```
S = 0.25, so 1 − S = 0.75

N = 2:   1 / (0.25 + 0.75/2)
       = 1 / 0.625   = 1.60

N = 4:   1 / (0.25 + 0.75/4)
       = 1 / 0.4375  ≈ 2.29

N → ∞:   1 / 0.25    = 4.00
```



Going from 8 to 16 cores (double the hardware) only raises the speedup from 2.91 to 3.37. Reduce the serial part first — then add cores.

User-Level Threads vs. Kernel-Level Threads

Who knows that the threads exist — only a library in the program, or the kernel itself?

User-level threads

- Managed by a library inside the process; the kernel sees one single-threaded process.
- Creating and switching: just function calls — very fast, no system call.
- One thread makes a blocking system call → the kernel blocks the whole process.
- Cannot use two cores: the kernel schedules only one thing.

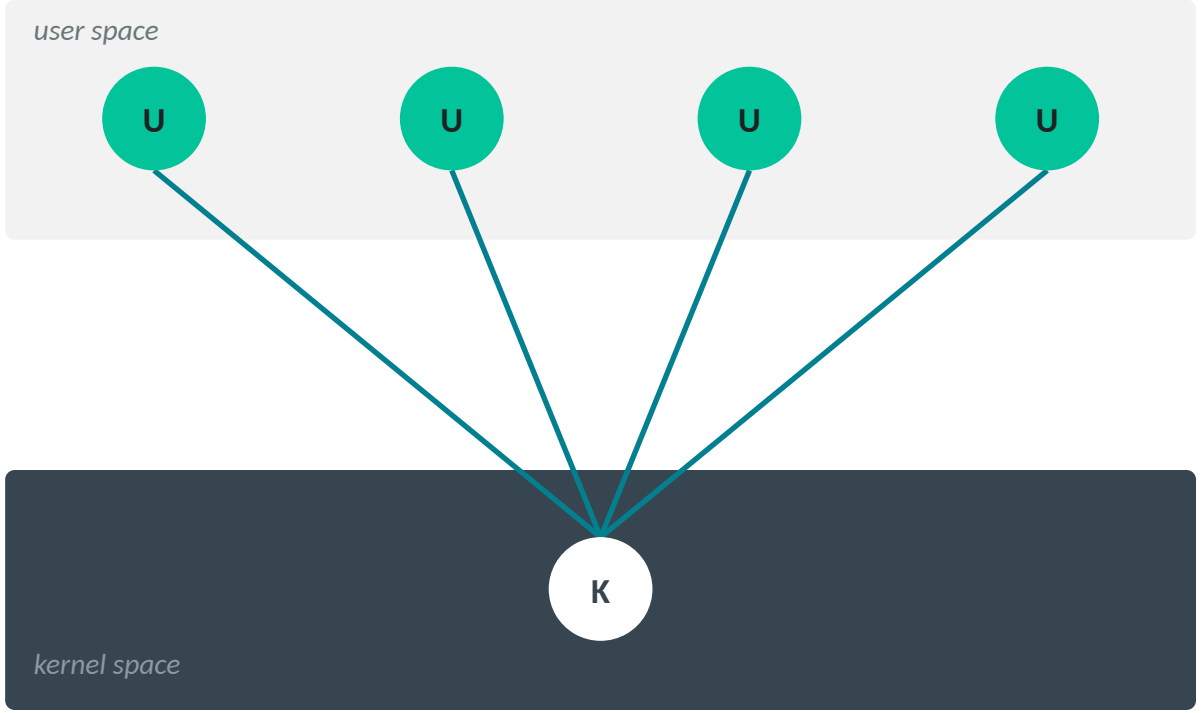
Kernel-level threads

- Created and scheduled by the OS kernel; each has its own kernel record.
- Creating and switching need system calls — slower than user threads.
- One thread blocks → the kernel runs another thread of the same process.
- Threads of one process run in parallel on several cores.

Every user thread must eventually run on some kernel thread. How they are matched is the multithreading model.

Model 1: Many-to-One

Many user threads are mapped onto one kernel thread. Thread management is done by a user-space library.



U = user thread, K = kernel thread

Advantages

- Fast: thread switching without the kernel.
- Works even on an OS without kernel threads.

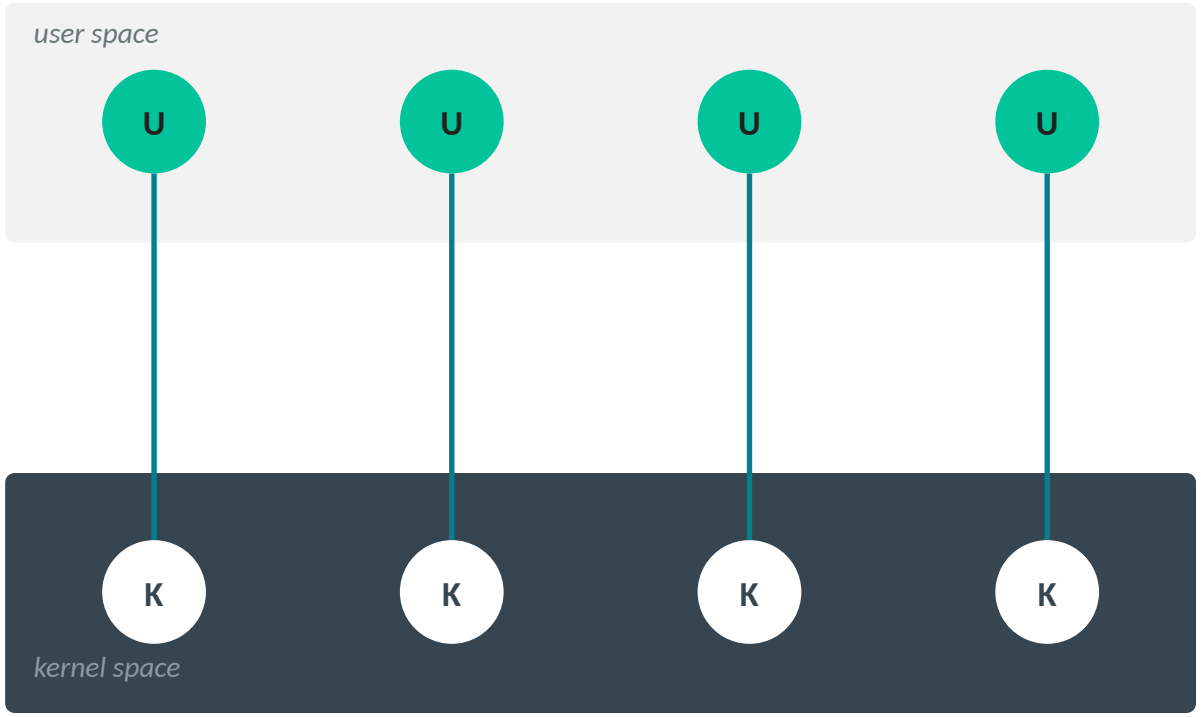
Disadvantages

- One blocking system call blocks ALL threads.
- No parallelism: only one thread can be in the kernel at a time.

Examples: Solaris Green Threads, GNU Portable Threads. Rarely used today.

Model 2: One-to-One

Each user thread is mapped onto its own kernel thread. pthread_create() asks the kernel for a new thread.



Each pthread_create() creates a new kernel-scheduled thread.

Advantages

- A blocking call blocks only that thread.
- True parallelism on multicore CPUs.

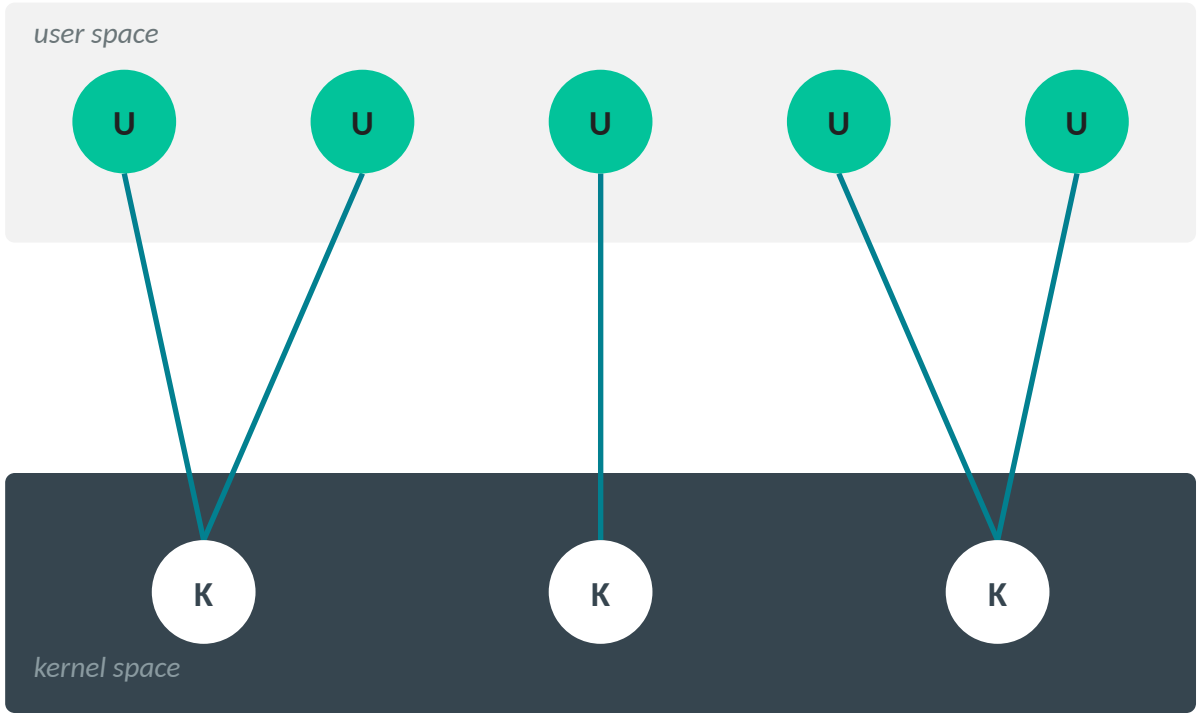
Disadvantages

- Each user thread costs a kernel thread: creation needs a system call.
- Very many threads can burden the kernel.

Used by Linux (NPTL) and Windows — the model we see in our programs today.

Model 3: Many-to-Many

Many user threads are multiplexed onto a smaller or equal number of kernel threads.



The library decides which user thread runs on which kernel thread.

Advantages

- Create as many user threads as you like, cheaply.
- Still parallel on multicore, and a blocking call does not stop everything.

Disadvantages

- Complex to implement: two schedulers (library + kernel) must cooperate.

Variant two-level model: also allows binding one thread 1:1. Modern idea: Go goroutines, Java virtual threads.

SUMMARY

The Three Models Side by Side

Model	User : kernel	Parallel on multicore?	Blocking call stops…	Used by
Many-to-one	N : 1	no	all threads	Green threads (old)
One-to-one	1 : 1	yes	only that thread	Linux, Windows
Many-to-many	N : M (M ≤ N)	yes	only that thread	Go, Java virtual threads

Linux and Windows use one-to-one: every thread you create with pthread_create() is a separate entity in the kernel's scheduler.

POSIX Threads (Pthreads): The Essential Calls

Three main thread libraries: POSIX Pthreads (Linux, macOS), the Windows thread API, and Java threads.

Call	What it does
pthread_create(&t, NULL, f, arg)	Start a new thread that runs function f(arg). Its handle is stored in t.
pthread_join(t, &result)	Wait until thread t has finished (like wait() for processes).
pthread_exit(value)	End the calling thread (returning from f does the same).
pthread_self()	Return the calling thread's own handle.
gcc -pthread prog.c	Compile and link with the Pthreads library.

Pthreads is a specification (POSIX standard), not an implementation. On Linux it is implemented by glibc (NPTL) on top of the clone() system call.

HANDS-ON

Example 2: Hello from Three Threads

```c
int shared_counter = 0;          /* global: ONE copy */

void *worker(void *arg)
{
    int my_number = *(int *)arg; /* on MY stack */
    printf("thread %d: PID = %d, TID = %d\n",
           my_number, getpid(), gettid());
    shared_counter++;
    return NULL;
}

int main(void)
{
    pthread_t tid[3];  int number[3];
    for (int i = 0; i < 3; i++) {
        number[i] = i + 1;
        pthread_create(&tid[i], NULL, worker, &number[i]);
    }
    for (int i = 0; i < 3; i++)
        pthread_join(tid[i], NULL);   /* wait */
    printf("shared_counter = %d\n", shared_counter);
}
/* full program: code/thread_hello.c */
```

What to notice

- All threads print the same PID: one process.
- Each thread has its own TID (a kernel thread: one-to-one).
- Threads finish in any order: 3 printed before 2!

```
$ ./thread_hello
main    : PID = 6351, TID = 6351
thread 1: PID = 6351, TID = 6352
thread 3: PID = 6351, TID = 6354
thread 2: PID = 6351, TID = 6353
main    : all threads done,
          shared_counter = 3
```

HANDS-ON

Example 3: Summing an Array with T Threads

```
int  *a;              /* the big array: shared  */
long  partial[MAXT];  /* one slot per thread    */

void *sum_slice(void *arg)
{
    long k = (long)arg;               /* my slice */
    long start = k * N / T, end = (k + 1) * N / T;
    long s = 0;
    for (long i = start; i < end; i++)
        s += a[i];
    partial[k] = s;                   /* my slot  */
    return NULL;
}

/* in main(): */
for (long k = 0; k < T; k++)
    pthread_create(&th[k], NULL, sum_slice, (void *)k);
for (long k = 0; k < T; k++) {
    pthread_join(th[k], NULL);
    total += partial[k];
}
/* full program (adds timing, 20 repeats): code/array_sum.c */
```

Data parallelism

- 100 million ints (400 MB), cut into T equal slices.
- Thread k sums slice k into partial[k] — no sharing of results, so no locks needed.
- main() waits for all threads (join) and adds the partial sums.

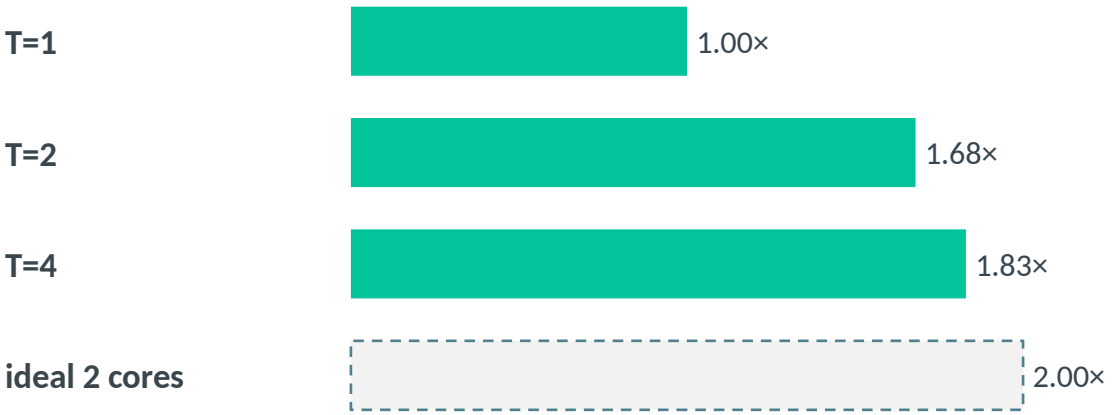
```
$ ./array_sum 1
threads = 1  sum = 9000000000  time = 1.686 s
$ ./array_sum 2
threads = 2  sum = 9000000000  time = 0.999 s
```

HANDS-ON

Measured: 1, 2 and 4 Threads on a 2-Core Machine

Median of 9 runs on our test VM ($ nproc → 2). The machine was shared with other jobs, so single runs varied by up to ±30 %.

Threads T	Time (median)	Speedup
1	1.564 s	1.00
2	0.932 s	1.68
4	0.855 s	1.83



Interpretation

- **2 threads:** 1.68× faster — good, but below the ideal 2×.
- **4 threads:** hardly better: there are only 2 cores, so at most 2 threads run at any instant.
- **Why not 2×?** creating and joining threads, sharing memory bandwidth, and other jobs on the machine.
- Amdahl: 1.68× on 2 cores behaves like S ≈ 0.19 — the "serial part" includes all such overheads.

HANDS-ON

Seeing Threads with ps and /proc

```
$ ./thread_hello &          # threads sleep 2 s
$ ps -eLf | grep thread_hello
UID    PID  PPID   LWP  C NLWP CMD
root 27241 27234 27241  0    4 ./thread_hello
root 27241 27234 27243  0    4 ./thread_hello
root 27241 27234 27244  0    4 ./thread_hello
root 27241 27234 27245  0    4 ./thread_hello

$ ls /proc/27241/task
27241  27243  27244  27245

$ grep Threads /proc/27241/status
Threads:  4
```

Reading the output

- **-L** shows one line per thread.
- **PID** is the same on all lines: one process.
- **LWP** (light-weight process) = thread ID.
- **NLWP** = number of threads: main + 3 workers = 4.
- /proc/<pid>/task/ has one folder per thread.

INTERMISSION

10-Minute Break

Next: the CPU scheduler — who runs next, and how do we measure 'good'?

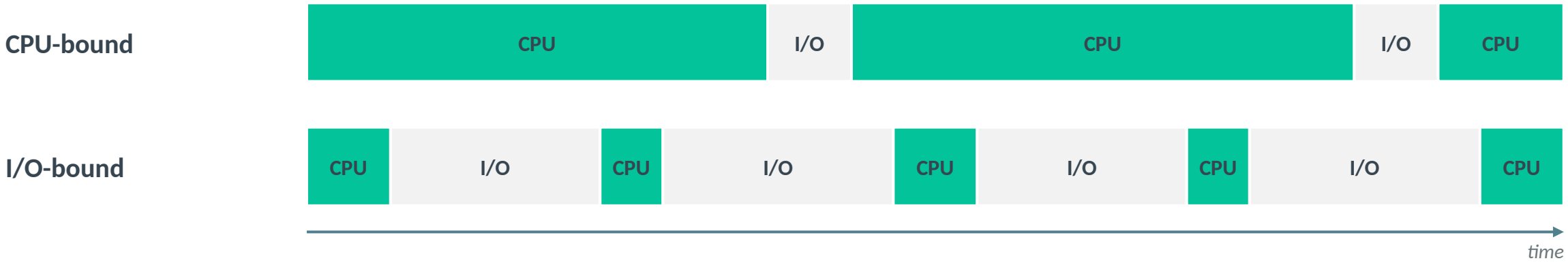
PART 2

CPU Scheduling: Basic Concepts

Many ready threads, few cores: someone must choose.

The CPU–I/O Burst Cycle

A process alternates between computing (CPU burst) and waiting for I/O (I/O burst).



CPU-bound

Few, long CPU bursts. Examples: video encoding, compiling, scientific simulation.

I/O-bound

Many short CPU bursts, mostly waiting. Examples: text editor, web server, file copy.

Measured fact

Most CPU bursts are short (a few ms); only a few are long. Schedulers are designed around this.

Example 4: CPU-Bound vs. I/O-Bound with time

```
/* ./burst cpu : one long CPU burst */
for (long i = 1; i <= 4000000000L; i++)
    x += 1.0 / i;

/* ./burst io : 1000 x (tiny CPU burst + I/O wait) */
for (int i = 0; i < 1000; i++) {
    buf[0] = 'A' + i % 26;     /* CPU    */
    write(fd, buf, 4096);      /* I/O... */
    fsync(fd);                 /* ...wait for disk */
}
/* full program: code/burst.c */
```

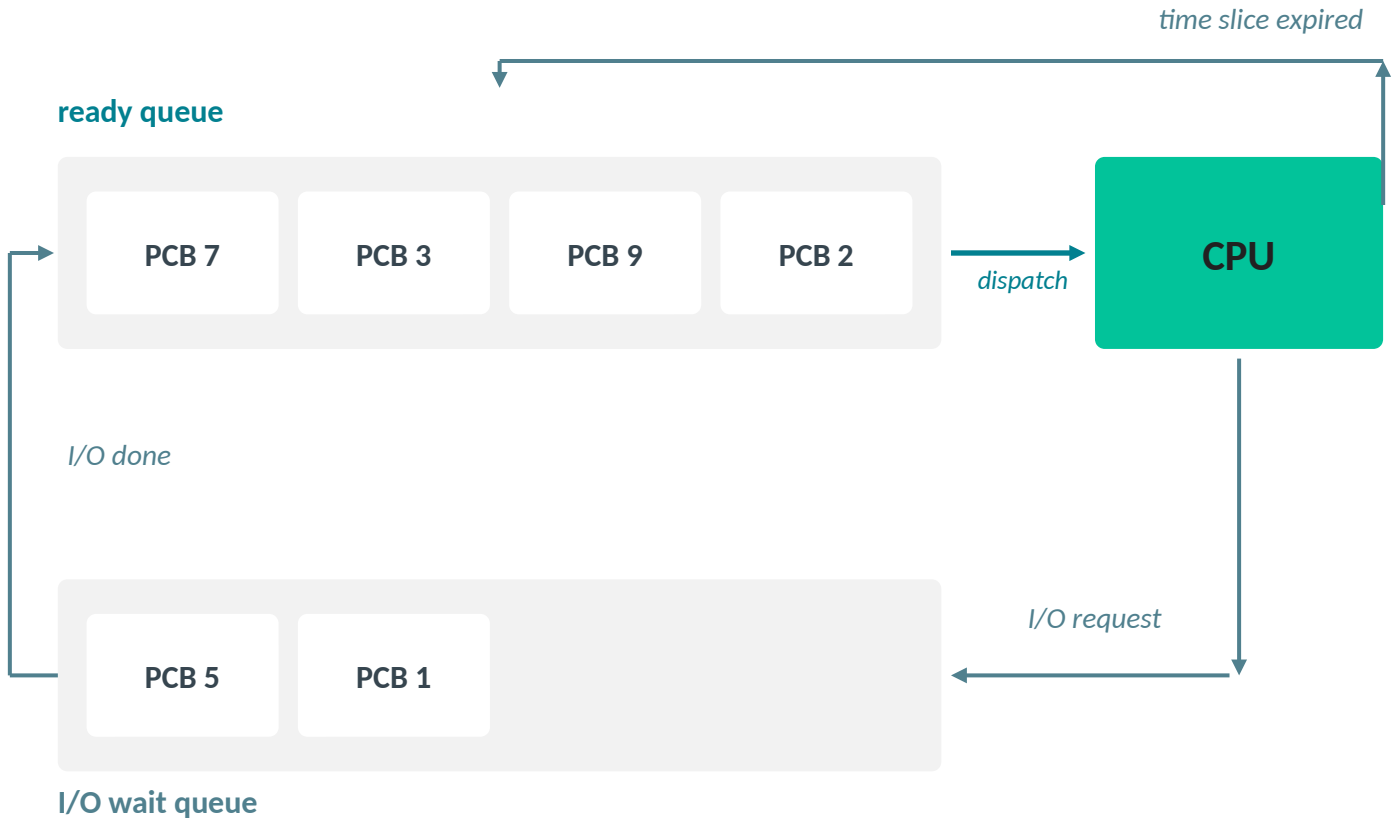
real vs. user + sys

- **real:** wall-clock time.
- **user / sys:** CPU time in user / kernel mode.
- CPU-bound: real ≈ user + sys.
- I/O-bound: real ≫ user + sys — about 73 % of the time the process was waiting, and the CPU was free for others.

```
$ time ./burst cpu
real 0m1.266s  user 0m1.197s  sys 0m0.001s
$ time ./burst io
real 0m0.384s  user 0m0.000s  sys 0m0.104s
```

The CPU Scheduler and the Ready Queue

Whenever the CPU becomes idle, the short-term scheduler selects one process from the ready queue.

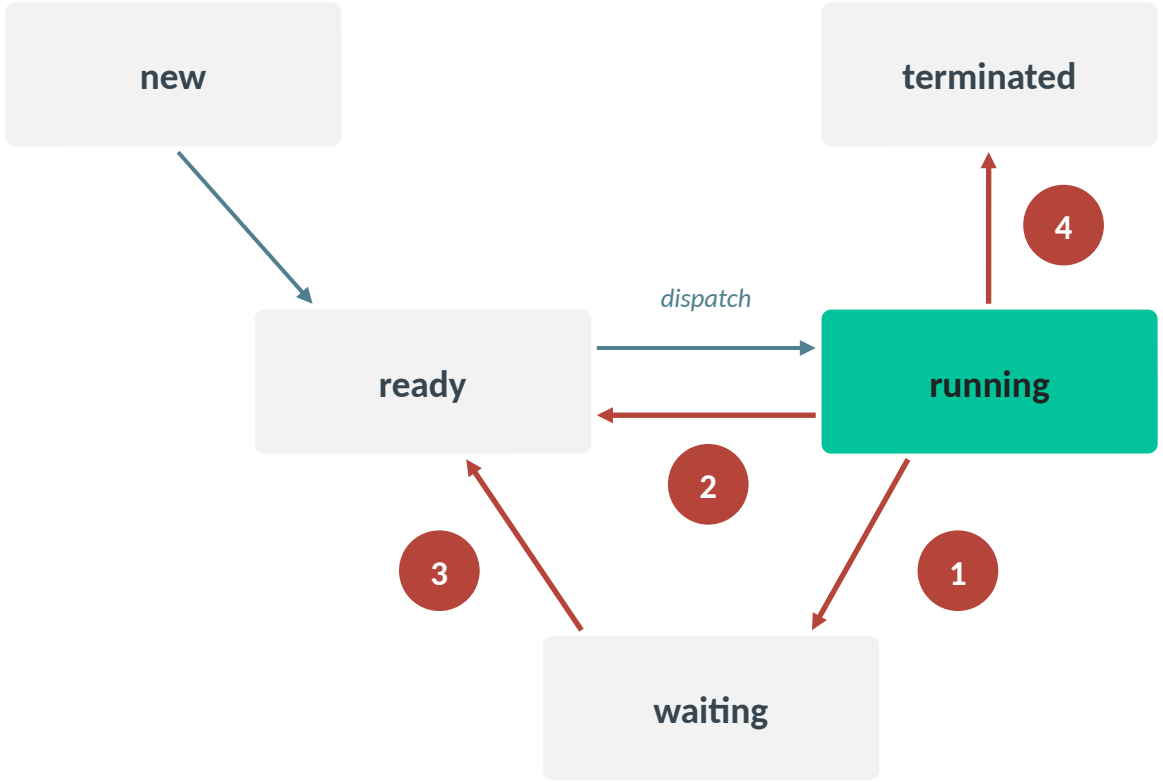


Key points

- The ready queue holds the PCBs of processes that could run right now.
- It is not necessarily FIFO: it may be a priority queue, a tree, or a list. Choosing the order is the scheduling algorithm (Week 5).
- The short-term scheduler runs very often (every few ms), so it must be fast.

When Does the Scheduler Have to Decide?

Four situations in the process-state diagram can trigger a scheduling decision.



The four situations

- **1. running → waiting:** I/O request, or wait() for a child.
- **2. running → ready:** an interrupt, e.g. the timer says the time slice is over.
- **3. waiting → ready:** the I/O has completed.
- **4. running → terminated:** the process exits.
- In 1 and 4 the CPU is free: a new process MUST be chosen. In 2 and 3 the OS MAY choose to switch.

Non-Preemptive vs. Preemptive Scheduling

Non-preemptive (cooperative)

- Scheduling only in situations 1 and 4.
- Once a process has the CPU, it keeps it until it blocks or terminates.
- Simple, but one long process can make everyone else wait.
- Windows 3.x, classic Mac OS (before OS X).

Preemptive

- Scheduling also in situations 2 and 3.
- The OS can take the CPU away (using the timer interrupt).
- Better responsiveness; a runaway program cannot freeze the system.
- All modern OSes: Windows, macOS, Linux, Android, iOS.

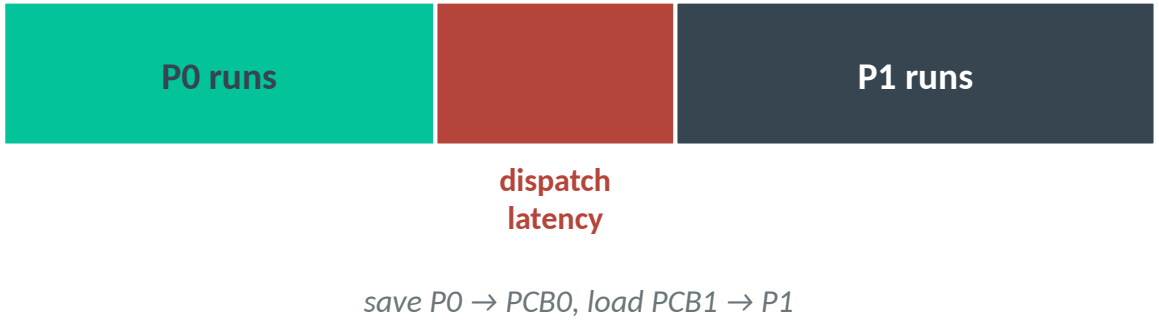
Price of preemption: a thread can be interrupted in the middle of updating shared data → race conditions. This is the second half of Week 5.

The Dispatcher and Dispatch Latency

The scheduler decides WHO runs next; the dispatcher actually gives that process the CPU.

The dispatcher's three jobs

- Switch context: save the old process's state into its PCB, load the new one's.
- Switch to user mode.
- Jump to the right place in the new program (its saved PC).



Dispatch latency is pure overhead: no useful work is done.

It is small (microseconds) but happens very often: watch vmstat 1 — the cs column on our idle test VM showed about 500–600 context switches per second, a busy server shows tens of thousands.

HANDS-ON

Counting Context Switches: Voluntary or Forced?

GNU time -v reports how often the process left the CPU itself (blocked) or was preempted.

```
$ /usr/bin/time -v ./burst cpu
  User time (seconds): 1.17
  System time (seconds): 0.01
  Elapsed (wall clock) time: 0:01.22
  Voluntary context switches: 1
  Involuntary context switches: 50

$ /usr/bin/time -v ./burst io
  User time (seconds): 0.00
  System time (seconds): 0.11
  Elapsed (wall clock) time: 0:00.46
  Voluntary context switches: 3714
  Involuntary context switches: 0
```

Reading the result

- **Voluntary:** the process blocked (situation 1) — typical of I/O-bound programs.
- **Involuntary:** the scheduler preempted it (situation 2) — typical of CPU-bound programs.
- 50 preemptions in 1.2 s: the CPU-bound program was interrupted roughly every 24 ms on average.

How Do We Judge a Scheduler? Five Criteria

Criterion	Definition	Goal
CPU utilisation	Fraction of time the CPU is busy doing useful work (40 % light load … 90 % heavy).	maximise
Throughput	Number of processes completed per unit of time.	maximise
Turnaround time	From arrival (submission) to completion: everything, incl. waiting and I/O.	minimise
Waiting time	Total time spent waiting in the ready queue. The only part the scheduler directly controls.	minimise
Response time	From arrival until the first time the process gets the CPU (first response).	minimise

Usually we optimise the average. For interactive systems a predictable (low-variance) response time can matter more than a low average.

The Definitions as Formulas

For a process with a single CPU burst (no I/O), all times are measured on the same clock.

$$T_{\textbf{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}$$

$$T_{\textbf{waiting}} = T_{\text{turnaround}} - T_{\text{burst}}$$

$$T_{\textbf{response}} = T_{\text{first run}} - T_{\text{arrival}}$$

burst = 3 + 2 = 5, waiting = 4 + 2 = 6, turnaround = 11



Check with the picture

- Turnaround = 11 − 0 = 11.
- Waiting = 11 − 5 = 6 (= 4 + 2 in the ready queue).
- Response = 4 − 0 = 4.
- With I/O bursts, waiting time = only the time in the READY queue; subtract I/O time as well.

WORKED EXAMPLE

Practice: Four Processes in Arrival Order (FCFS)

The "obvious" order — first come, first served — on one CPU, no preemption. Times in ms.

Process	Arrival	Burst
P1	0	5
P2	1	3
P3	2	6
P4	16	2

Week 5 asks the real question: is arrival order a good order? (Try swapping P2 and P3 and compare.)

Gantt chart



How to read it

- P1 arrives at 0 and runs at once. P2 and P3 arrive while P1 runs: they wait in the ready queue.
- At 14 the ready queue is empty: the CPU is idle until P4 arrives at 16.
- Next slide: compute every criterion from this chart.

WORKED EXAMPLE

Practice: Computing the Criteria

Process	Arrival	Burst	Completion	Turnaround = C − A	Waiting = TAT − B	Response
P1	0	5	5	5 − 0 = 5	5 − 5 = 0	0
P2	1	3	8	8 − 1 = 7	7 − 3 = 4	4
P3	2	6	14	14 − 2 = 12	12 − 6 = 6	6
P4	16	2	18	18 − 16 = 2	2 − 2 = 0	0
Average				26 / 4 = 6.5 ms	10 / 4 = 2.5 ms	2.5 ms

CPU utilisation = 16 ms busy / 18 ms = 88.9 %. Throughput = 4 processes / 18 ms ≈ 0.22 per ms (222 per second).

Which Criterion Matters Most? It Depends

Maximise CPU utilisation and throughput; minimise turnaround, waiting and response time. The goals conflict.

Batch / servers

- Throughput and CPU utilisation.
- Long jobs are fine as long as the total work done per hour is high.

Desktop / phone

- Response time: the user must see a reaction at once.
- Predictability: a constant 50 ms feels better than 5 ms that sometimes becomes 2 s.

Real-time

- Meeting deadlines matters more than any average.
- Late = wrong (brakes, drones, medical devices).

Trade-off example: switching very often improves response time but adds dispatch overhead and lowers throughput.

TRY IT YOURSELF

Threads and Scheduling on Your Own Machine

```
# Linux (lab PCs, WSL, or a virtual machine)
$ nproc                    # how many cores?
$ make                     # in the code/ folder
$ ./array_sum 1; ./array_sum 2; ./array_sum 4
$ ./thread_hello & ps -eLf | grep thread_hello
$ time ./burst cpu ; time ./burst io
$ vmstat 1 5               # cs = switches/s

# Windows
# Task Manager > Details > add column "Threads"
# Task Manager > Performance > CPU
```

Questions to answer

- How many cores does your machine have? What speedup do you get with 2, 4 and 8 threads?
- Use Amdahl's law to estimate S from your 2-thread speedup.
- Which program on your PC has the most threads?
- Is your browser CPU-bound or I/O-bound while you read a page? And while it plays a video?

Week 4 — Key Takeaways

- **Thread:** a unit of execution with its own PC, registers and stack; threads of a process share code, data, heap and files.
- **Benefits:** responsiveness, resource sharing, economy (≈ 4× cheaper to create here), scalability on multicore.
- **Amdahl's law:** speedup ≤ 1 / (S + (1 − S)/N); the serial part S sets the limit 1/S.
- **Models:** many-to-one, one-to-one (Linux, Windows), many-to-many; Pthreads: pthread_create / pthread_join.
- **Scheduling basics:** CPU–I/O bursts, ready queue, four decision points, preemptive vs. non-preemptive, dispatch latency.
- **Criteria:** maximise utilisation and throughput; minimise turnaround (C − A), waiting (TAT − B) and response time.

QUICK QUIZ

Check Your Understanding

1. Two threads of the same process: name two things they share and two things each has privately.

2. A program is 90 % parallel. What is the maximum speedup on 4 cores? And on infinitely many cores?

3. Why can a many-to-one thread library not use a second CPU core?

4. P1 (arrival 0, burst 4) and P2 (arrival 1, burst 2) run FCFS. Find the turnaround and waiting time of each.

NEXT WEEK

Scheduling Algorithms & Synchronisation

Week 5 will cover:

- Scheduling algorithms: FCFS, SJF / SRTF, Priority, Round Robin
- Gantt charts and average waiting times for each algorithm
- Race conditions and the critical-section problem
- Mutex locks and semaphores

QUESTIONS?

Discussion

If you have questions after class, ask them on the course Telegram channel — everyone benefits from the answer.

Asst. Prof. Dr. Hasan Oğuz